

Computer Graphics: Lecture 2

A brief History Real-time 3D

And what we're going to cover in class

Slide Deck © Grant Williams, 2026, License: [CC-BY-SA 4.0](#)

Welcome Back!

- Last lecture we learned the expectations of the class
- We learned about the importance of the GPU
- We saw briefly what we'd be learning [any reminders?]
- We learned which technologies we will use [can anyone give me a list?]
- We learned why it will be valuable to learn to program the GPU. It's more than just video games! (but the video games are cool too) [what other things can it do?]

This lecture:

We're going to learn context:

- We're going to see the unique requirements and motivations behind 3D graphics development, framed in their historical context.
- We're going to learn how 3D graphics turned into a radically different paradigm than ordinary software development.
- We're going to learn why that matters: the 3D graphics pipeline.

The pipeline

This pipeline concept is incredibly important.

We don't program 3D apps the same way as other apps. Everything you have learned on your computer science journey up to now will be useful, but you will need to go beyond basic synchronous programming.

But it seems very strange at first, so I want to give you some context about the field and some ancient history to motivate it.

Computer Graphics

As a field, computer graphics is the study of generating images with computers. That's it.

This seems like a very simple definition, and it is, but the field is anything but simple.

Humans who can see often use it as their primary sense: most of us are very visual creatures. Therefore, we are very demanding of the images that we look at.

Specifically, we are demanding in two ways...

What we demand of computer graphics

1. We demand that the images have certain qualities (realism, artistic style, clarity, etc.)
2. We demand that they be generated quickly.

Let's talk first about the qualities we want...

Kinds of generated images

There are many applications for computer graphics. The application determines what qualities we want the generated images to have:

- Scientific visualization: the key is on expressing mathematical concepts to humans quickly. So images need to be clear and understandable.
- Simulation: realism is important here.
- Art (including games): creative expression is important

These are jobs that exist. Someone has to program the graphics card to achieve the desired visual effects. Look for job titles such as “graphics engineer” or “technical artist”.

Computer Graphics is a Broad Term

Any image generated by a computer is technically “computer graphics”

- Your operating system’s desktop is computer graphics
- Photoshop is computer graphics
- A movie being played on a computer is computer graphics
- A 3D game is computer graphics

The latter two are especially important.



Games and Movies

There are basically two “modes” in which we generate 3D graphics:

- **Pre-rendered:** we know what images we will need in advance, such as the frames of a movie. We have time to **render** (draw) these images, so that when we're ready, they're easy to display on the screen. Obviously, faster is better, but it can take hours and still work.
- **Realtime:** we *don't* know the images we will need in advance. They depend on something, such as user input or randomness. This is the case for video games.

What do I mean by *not knowing* the images in advance?

A video game “frame”

Consider this game screenshot:

It was only useful for 1/30th of a second.

It reflected the exact position of mario, the exact orientation of the camera, the exact position and animation of the “chain chomp” (the toothy enemy), his little chain “spheres”, the number of lives, etc.



A video game “frame” (2)

A single image that is part of a larger animated series is called a **frame**.

That frame was produced in response to extremely precise circumstances.

If the camera had been slightly different, if the player had approached the enemy slightly differently, if they had a different number of lives or amount of health, the image would be different.

If it takes too long (more than 1/30th of a second) to make the image, the game becomes sluggish and hard to control.

Performance is critical

Therefore performance is absolutely critical.

That game was for the Nintendo 64, by modern standards a very primitive system. And yet, it was able to produce those images (somewhat) quickly using specialized hardware.

Using this hardware requires special programming techniques.

So now, let's talk history. How have these techniques and hardware capabilities emerged?

Questions?

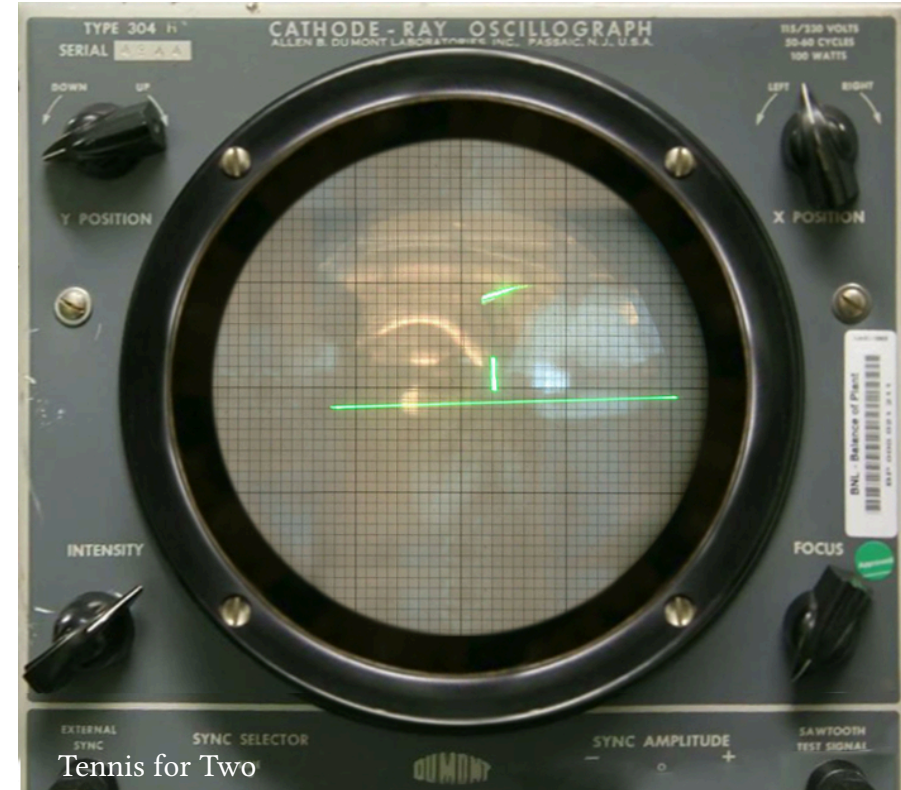
In the beginning (1958)

(Probably) the first video game was a game called Tennis for Two

A 2D oscilloscope was used to plot a ball, paddles, and a net.

OpAmps were used to store the position of the ball, as well as to modify it according to velocity and gravity.

It was an analog computer game.



Really, an oscilloscope?

An oscilloscope seems like a strange choice for a display technology, but there really weren't other choices.

Televisions existed, but generating a TV frame from something other than a recording would have required specialized hardware that didn't exist yet.

Therefore, an oscilloscope that could draw a little dot based on an input voltage was perfect.

The net, ground, and paddles were just a “dot” that was swept over a line. The “ghosting” made a solid-ish line.

Cathode Ray Tubes

The most common screen technology then was the Cathode Ray Tube.

An “electron gun” would shoot electrons in a beam onto a phosphor-coated screen.

The specific voltage and frequency would generate color which would continue to be visible for some milliseconds.

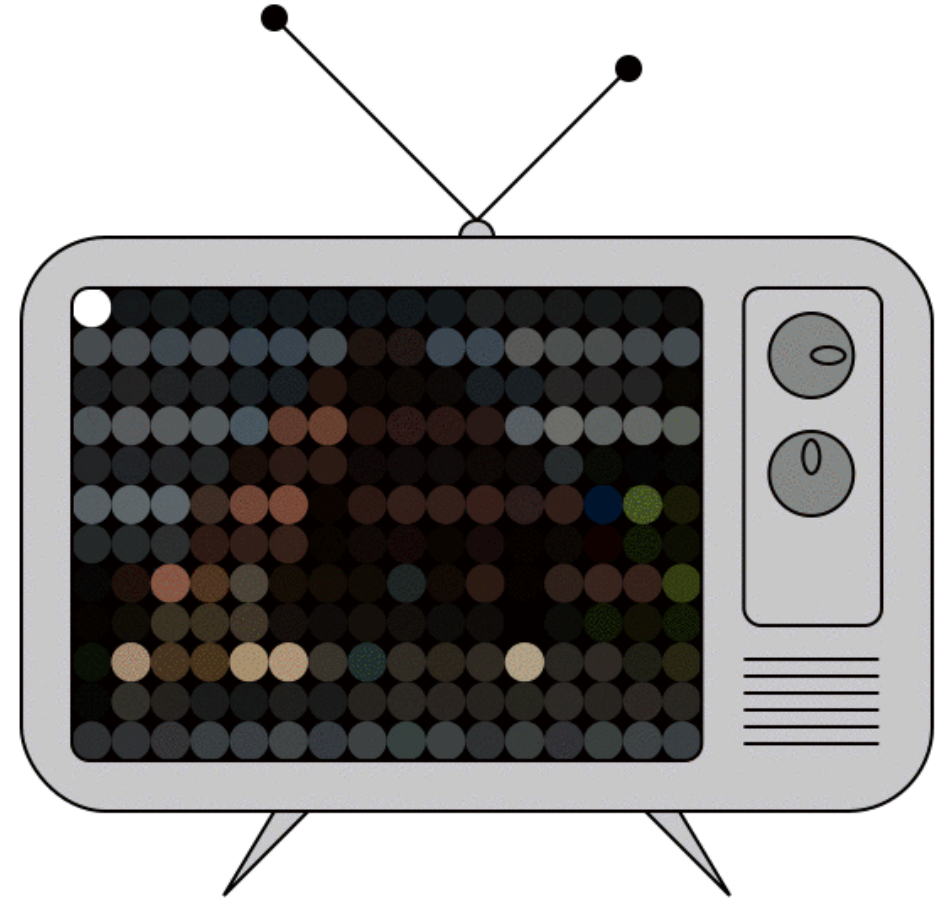


A partially disassembled CRT (Credit: [Raymond Spekking](#), CC-BY-SA 4.0)

The electron gun

There were basically two kinds of CRT:

1. Raster displays: the electron gun followed a fixed path.
2. Vector displays: the electron gun could be repositioned between draws.



An animation showing how a single frame was generated on a raster CRT. By [Laserlicht](#), [CC-BY-SA 4.0](#)

SA 4.0

Vector Displays